

FitBuddy – AI Fitness Plan Generator using Gemini Models

3. AI Model Selection & Architecture

Perform well in real-time, web-based interaction

Model Evaluation

We initially experimented with several Hugging Face models:

T5 & BART – For text generation (workout/nutrition)

DistilBERT – For potential intent classification

Bloom – For general purpose generation

While these models showed promise in basic text generation, they had notable limitations.

These models required:

Hosting with Hugging Face or local GPU setup

Fine-tuning or complex prompting to achieve structured plans

Final Model Selection

After comparative testing, we selected Google Gemini models for production:

Gemini 1.5 Pro → Used for structured 7-day workout generation and plan updating → Delivers rich, context-aware, formatted responses reliably

Gemini Flash → Used for generating fast, practical nutrition tips → Lightweight and efficient for quick API calls

Reasons for Choosing Gemini:

API-based, easy to integrate with FastAPI

Naturally understands structured formatting (days, sections)

High accuracy with minimal prompt tuning

Scalable and fast for feedback-based revisions

Activity 1.2. Define the architecture of the application

Architecture Overview: FitBuddy follows a modular FastAPI-based architecture-

Frontend: HTML + Jinja2 templates for user input and display

Backend: FastAPI for route handling and logic

AI Layer: Google Gemini APIs (Pro & Flash)

Database: SQLite with SQLAlchemy for persistence

Frontend Responsibilities: Users interact via-

index.html – Input form

result.html – Plan, tip, feedback

all_users.html – Admin panel

Backend Responsibilities:

Receives form data

Calls Gemini models

Stores user info and plans

Renders dynamic templates

AI Integration Points:

Workout generation & feedback updates → Gemini Pro

Nutrition tips → Gemini Flash

Activity 1.3 Set up the development Environment:

Install Python & Pip: Ensure Python and pip are installed.

Create Virtual Environment: `python -m venv fitbuddy-env` `fitbuddy-env\Scripts\activate` (Windows)

Install Required Libraries: `pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai`

Run Server: `uvicorn app.main:app --reload` Visit `http://127.0.0.1:8000` for the app and `/docs` for API testing.

Set Up Project Structure: Create folders: `/app` (Python files), `/templates` (HTML), `/static/images` (assets)

Milestone 2. Core functionalities Development

Activity 2.1 Develop the core functionalities:

Workout Plan Generation

Function: `generate_workout_gemini()` File: `gemini_generator.py`

Implements the core logic to generate a personalized 7-day workout plan using user inputs such as goal (e.g., muscle gain, weight loss) and intensity (low, medium, high). The prompt is sent to the Gemini 1.5 Pro model, which returns a structured, day-wise plan including:

Warm-up (5–10 mins)

Main workout (exercise details, sets, reps)

Cooldown or recovery tip

The output is returned to `routes.py` and rendered on the `result.html` page using `<pre>` blocks for formatting.

Nutrition Tip Generation

Function: `generate_nutrition_tip_with_flash()` File: `gemini_flash_generator.py`

This function generates a concise, practical nutrition or recovery tip tailored to the user's selected goal. It utilizes the Gemini Flash model to deliver a fast, helpful response such as hydration advice, protein suggestions, or general recovery best practices. This tip is displayed beneath the workout plan in `result.html`.

Feedback-Based Plan Updating

Function: `update_workout_plan()` File: `updated_plan.py`

This function enables the user to revise their original plan by submitting feedback (e.g., "Add yoga", "Include more cardio"). The original plan and feedback are passed to Gemini 1.5 Pro, which returns a modified plan with the suggested updates. The updated plan is then stored and displayed alongside the new nutrition tip.

User & Plan Storage

Functions:

`save_user()`, `save_plan()`, `update_plan()`

`get_original_plan()`, `get_user()`

File: `database.py`

All user information (ID, name, age, weight, goal, intensity) and AI-generated plans are stored in a SQLite database via SQLAlchemy ORM. Feedback-based updates are saved as a separate field to preserve both versions (original and updated).

Admin View of All Users

Route: /view-all-users Template: all_users.html

Displays a list of all users and their respective plans. Rendered with Jinja2 using a {% for user in users %} loop, it allows administrators to see original and updated plans clearly in a structured table format.

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define Routes in FastAPI

All routing logic is handled in routes.py. Each route is linked to the corresponding function developed in Activity 2.1.